

가능성을 기회로 바꾸고, 제품으로 가치를 전하는 메이커 김대정입니다.

기획자로 시작해 백엔드로 왔고, 지금은 아이디어를 제안한 AI 콘텐츠 제품의 백엔드·AI 실행부를 직접 만들고 핵심 화면은 coding agent로 완성해 월 1천만원 수준의 구독 매출이 발생하는 제품으로 운영합니다. 대규모 콘텐츠 데이터를 검수 가능한 구조와 평가 워크벤치로 바꿨고, 시술 지식의 hybrid retrieval API와 제품 기능을 대화로 제어하는 Agent PoC를 설계·검증했습니다.

미리디에서 먼저 확인할 네 가지

01 아이디어를 고객이 구독하는 AI 제품으로

기획·QA·마케팅과 고객 불편을 기능 우선순위와 생성 품질 기준으로 구체화하고, FastAPI 백엔드와 생성 품질 시스템을 직접 만들어 구독 제품으로 출시·운영했습니다.

02 검수 가능한 데이터 운영 체계

SNS 관측 데이터를 독립 schema·먹등 importer·사람 평가 워크벤치로 바꿨고, 시술·제품·안전 규칙·문헌 지식은 canonical 데이터와 release 단위 publication gate로 등록·검수·공개를 통제했습니다.

03 AI 기능 PoC와 품질·실험 흐름

제품 기능을 대화로 제어하는 planner-executor Agent PoC와, 구조화 조회가 판단하고 문헌 검색이 근거를 보장하는 hybrid retrieval API를 설계했습니다. 대표 예시 몇 개가 아니라 검수된 fixture로 recall·안전 규칙·응답 시간을 자동 측정하는 평가 게이트를 두었습니다.

04 빠른 배포와 품질 보장

운영 중인 제품을 멈추지 않고 backend를 교체해 결함 재발을 약 94% 줄였고, 실제 사용자가 쓰는 제품의 배포·QA·운영을 계속 전담하고 있습니다.

아이디어를 팀과 실제 고객이 구독하는 제품으로 만들고 운영에 필요한 시스템을 직접 구축했습니다.

고객이 돈을 내는 이유를 기획·QA·마케팅과 함께 구체화하고, 제품 판단부터 출시·운영까지 이끌었습니다. 제품에 필요한 Next.js 핵심 흐름을 직접 구현하고, 빠른 기능 검증 중심의 초기 백엔드를 인계받아 팀이 운영할 수 있는 FastAPI 구조로 재구축했습니다. 이어 제품 기준 데이터와 AI 실행 상태의 관리 책임을 나눴습니다. 팀과 함께 **구독료를 내는 고객이 쓰는 제품**으로 만들었습니다.

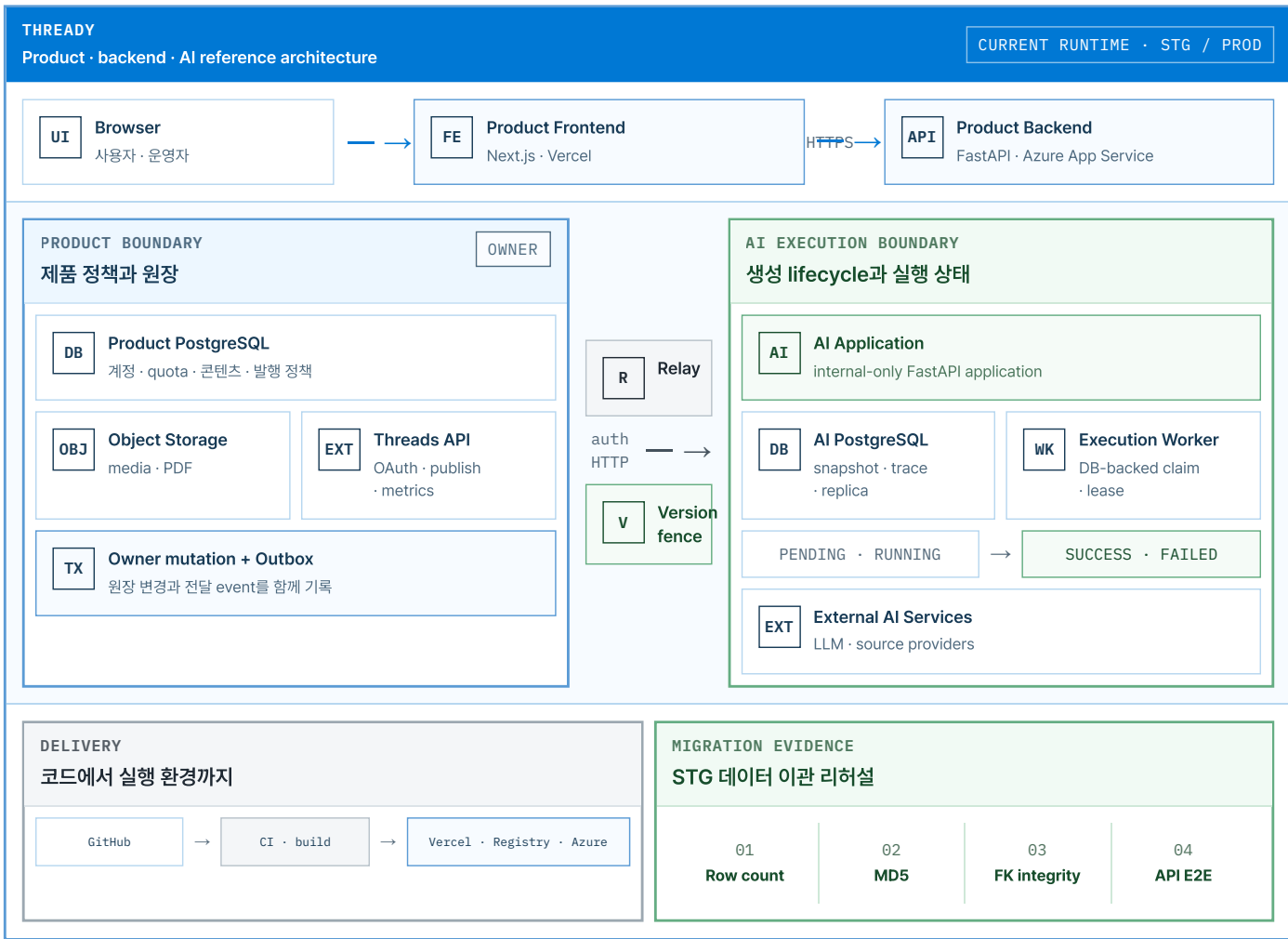
이 버전에서 보는 지점

Threads 글 제작의 반복을 제품 흐름으로 바꾸고 품질 판정을 자동 게이트·실측 분포·사람 판정 3층으로 나눈 판단, 그리고 SNS 관측 데이터를 독립 labeling schema·역등 importer·super-admin 평가 워크벤치로 바꾼 데이터 운영 체계를 봅니다.

CURRENT · SUBSCRIPTION 무료 제품 운영 구독 결제 · 제품 운영	NEXT · ADVERTISING 광고 수익 모델 검증 적용 시작 · 운영 데이터 수집 중
--	---

제품 운영과 실제 실행 환경을 한 구조로 연결했습니다.

PRODUCT · RUNTIME ARCHITECTURE



리드 제품 운영 · 우선순위 · 품질 · QA · 릴리스	직접 구현 제품 핵심 흐름 · Next.js · FastAPI · 데이터 · AI	팀 협업 기획 · QA · 마케팅 · 디자인
---	---	--

한 모델의 장애가 전체 생성 기능 중단으로 번지지 않게 운영했습니다.

외부 AI 장애 대응

감지	격리	작업 지속
Sentry 알림	문제 모델 일시 제외	정상 모델 선택
반복되는 외부 AI 5xx를 운영자가 확인	오류가 반복되는 모델을 사용자 선택지에서 제거	서비스 중인 다른 모델로 생성을 계속 진행

5xx는 재시도 가능한 실패로 분류하고 최대 시도 뒤 최종 실패 상태와 사용자 재시도 경로를 남깁니다. 당시 사용자-트래픽 규모에서는 복잡한 자동 복구보다 운영자가 문제 모델만 격리하는 방식이 적절하다고 판단했습니다.

공개 콘텐츠 분석을 제품 판단과 생성·평가 실험에 연결했습니다.

콘텐츠 데이터

라벨링 도구

Threads 공개 게시글과 작성자의 이어쓰기를 독립된 라벨링 영역으로 정리했습니다. 같은 자료를 다시 가져와도 중복되지 않는 가져오기 도구와 API-평가 화면을 함께 만들었습니다.

잘되는 글의 기준

최근 1년 내 게시된 콘텐츠를 중심으로 첫 문장·구조·어체·마무리의 관측 기준을 만들었습니다. 이 기준은 글 생성 프롬프트와 평가 실험에 반영해 결과를 비교했습니다.

인계받은 초기 백엔드를 팀이 운영할 수 있는 FastAPI 구조로 재구축했습니다.

백엔드 재구축

무엇이 문제였나 · 인계받은 초기 백엔드는 빠른 기능 검증 중심으로 만들어져 있었습니다. 기능은 돌아갔지만 어떤 코드가 무엇을 책임지는지, 수정 영향이 어디까지 퍼지는지 설명하기 어려웠습니다.

어떻게 풀었나 · 기존 API와 기능을 먼저 목록으로 만들고, 현재 동작을 확인할 테스트와 검증 기준을 세웠습니다. 프론트엔드는 그대로 둔 채 새 백엔드를 옆에서 만들었고, AI는 코드 분석과 반복 구현에 활용했습니다. 아키텍처와 검증 기준, 작업 범위, 전환 시점은 직접 판단했습니다.

무엇이 달라졌나 · 기존 백엔드와 새 백엔드의 응답을 비교한 뒤 전환했습니다. 전환 전후 같은 기준으로 비교했을 때, 해결된 QA 항목의 재오픈 비율은 26%p 낮아졌습니다. 이후 배포와 운영도 계속 말았습니다.

기존 서비스와 나란히 만든 뒤 전환		
유지	기존 API	사용자 흐름은 그대로
정리	기능 목록	기존 동작과 응답 확인
재구축	FastAPI 백엔드	AI로 분석·반복 구현 보조
기존·신규 동작 비교		→ 전환 결정

제품 원장과 AI 실행을 나누고, 중복·지연·역순 전달에도 최신 상태로 수렴하도록 설계했습니다.

BE-AI 분리 설계

01 · OWNER TX	02 · RELAY	03 · VERSION FENCE	04 · AI STATE
원장 변경 + Outbox	retry	최신 상태 보호	별도 application · DB
제품 원장 변경과 전달할 event를 같은 transaction에 기록	일시적인 전달 실패는 원장 transaction과 분리해 다시 시도	늦거나 역순인 변경이 더 최신 상태를 덮지 않도록 차단	생성 lifecycle과 실행 상태는 AI 경계가 소유

설계

3층 · 천장

이 글이 통하는가

사람 (합의 채점)

자동화하지 않는다. 판정 질문만 고정

2층 · 분포

플랫폼다운가

코드 (실측 분포 대조)

직접 수집한 corpus의 분포 스냅샷과 비교

1층 · 바닥

틀리지 않았는가

코드 (결정적 게이트)

하나라도 걸리면 재생성. 수치 날조·어체·접힘 등

측정 규율 — 점수는 도달점이 아니라 현 기준의 스냅샷

규율	언제	하는 일	통과	여기면	기록
축 먼저	실험 시작 전	어느 축을 올리려는 실험인지 먼저 말한다	축이 있는 실험	실험하지 않는다	축별 점수 이력
근거 붙이기	채점할 때	축마다 왜 이 점수인지, 뭘 고치면 오르는지, 그 근거	게이트 수차·원문 사례·실측 대조가 붙은 점수	점수만 있는 채점 무효	자가 채점 / 합의 채점 구분
기준 재실측	기준값을 쓸 때	기준이 실측인지 자사 출력의 되먹임인지 확인	실측 corpus 기준	자기 과교정을 목표로 삼는 순환	기준 변경 시 이전 점수와 직접 비교 금지
반증 로그	접근이 실패했을 때	지우지 않고 「다시 시도하지 말 것」에 남긴다	실험 범위가 좁아진다	같은 실패 반복	반증된 규칙은 자기 규칙이라도 제거

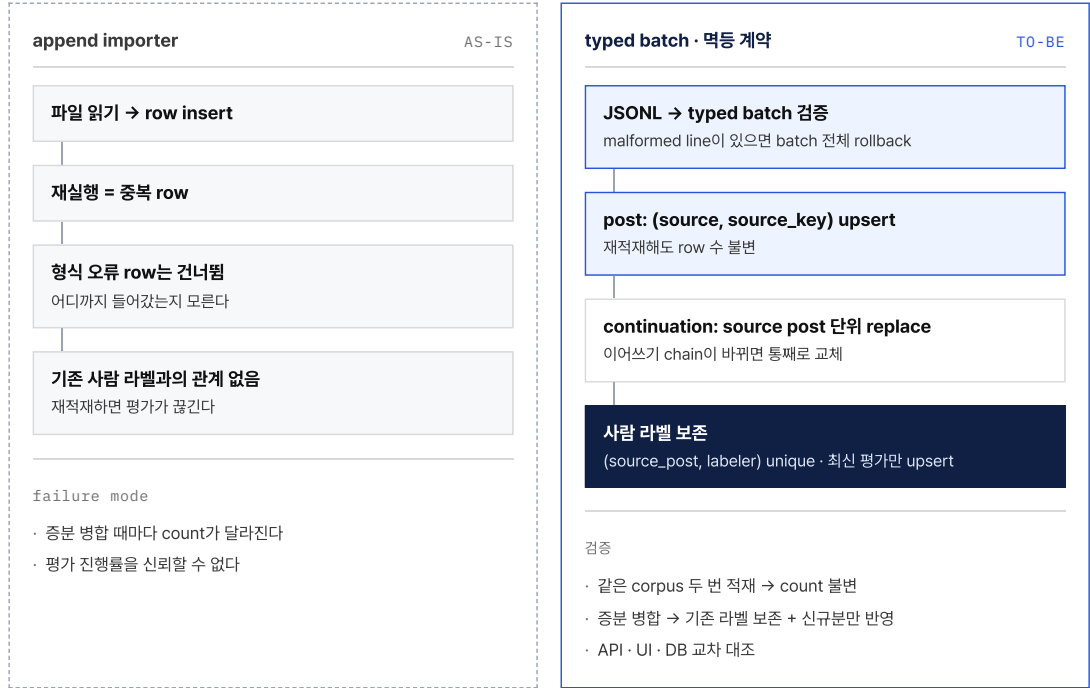
- 불변 조건

아래층을 못 넘으면 위층을 묻지 않는다. 자동화는 바닥에만 둔다.
- 버린 대안

글자 수 상한 지시 (형태를 바꿔도 과교정을 유발 → 분포·총량 비례로 전환) · 자사 출력 값을 기준으로 삼는 것 (재실측으로 발견해 교정)
- 검증 근거

6축 점수 이력 · 소표본에서 대량 corpus로 측정 확대하며 유지·기각 결론 구분 · 판정 책임을 writer에서 planner로 옮긴 실측 (정보를 가진 자리로 책임 이동)

아래층을 통과하지 못하면 위층을 묻지 않는다. 자동화가 닿는 층과 닿지 않는 층을 처음부터 갈라 설계했다.



여러 화면에 흩어진 Thready 기능을 대화 하나로 제어하는 방식을 검증했습니다.

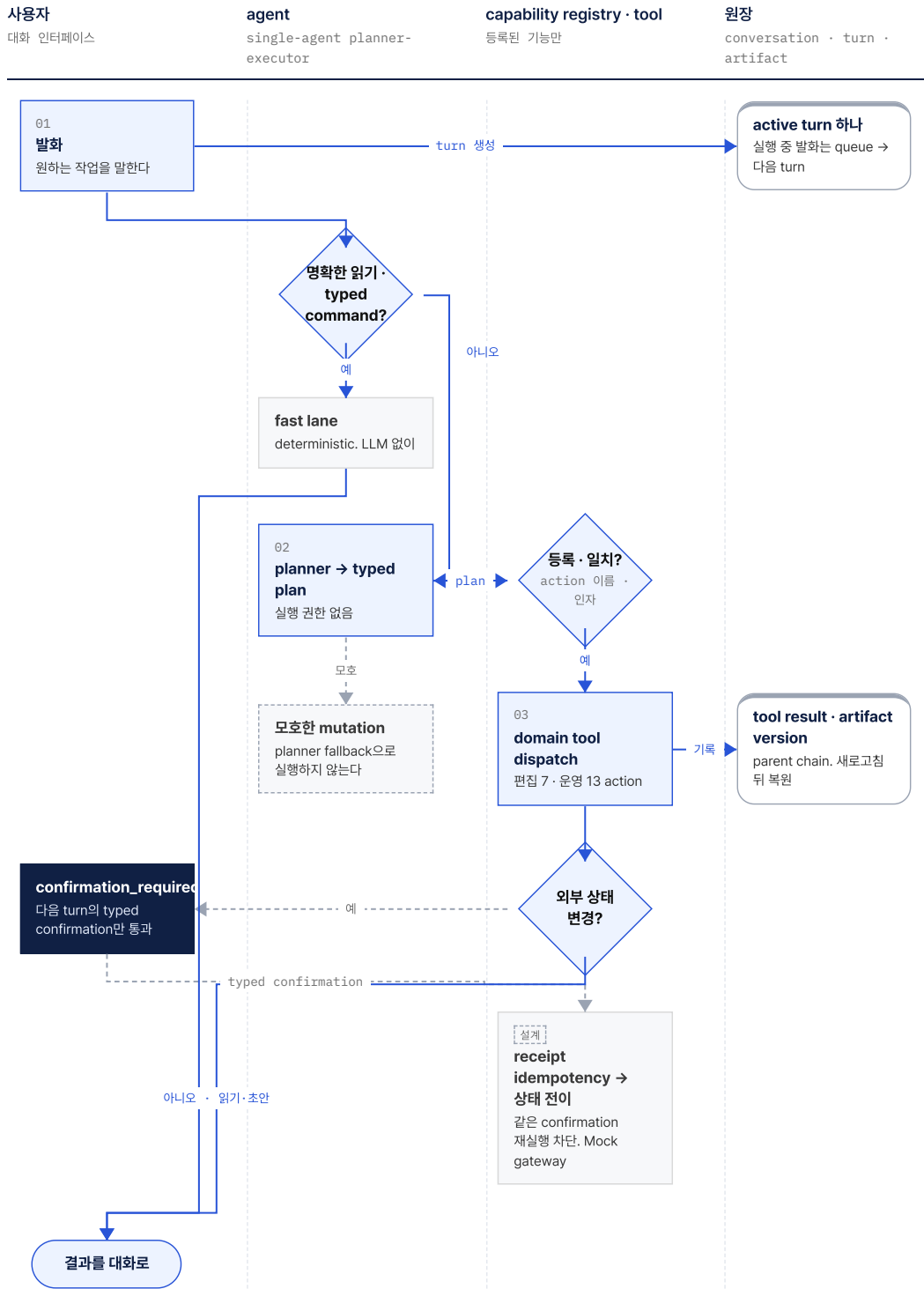
사용자가 기능의 위치와 실행 순서를 외우는 대신, 원하는 작업을 말하면 Agent가 필요한 Thready 기능을 선택하고 결과를 대화로 이어서 보여주는 독립 프로토타입을 만들었습니다.

이 버전에서 보는 지점

multi-agent 대신 하나의 **planner-executor**로 두고, **planner**는 **typed plan**만 만들고 실행 권한은 **capability registry**가 확인한 뒤 **dispatch**합니다. 예약·발행·삭제 같은 변경은 다음 **turn**의 **typed confirmation**을 통과할 때만 실행되고, **receipt** 기반 **idempotency**로 중복 실행을 막았습니다. 외부 상태 변경은 **Mock gateway**에서만 검증했습니다.

확인할 근거

- 콘텐츠 제작·계정 및 게시물 관리·성과 조회를 하나의 대화 흐름으로 구성
- 실행 계획과 등록 **capability**의 실행 권한 분리
- 대화·실행 요청·도구 결과·결과물 버전 기록
- 사용자 확인과 중복 실행 방지를 거치는 **Mock** 상태 변경
- 운영 Agent가 아닌 독립 프로토타입으로 범위 명시



불변 조건	planner는 실행 권한을 갖지 않는다. 등록되지 않은 action은 실행되지 않는다. 외부 상태 변경은 typed confirmation과 receipt를 통과할 때만 일어난다.
버린 대안	multi-agent orchestration (범위 대비 복잡) · 최근 message 개수 절단 (token-aware context selection + append-only compaction snapshot으로 대체)
검증 근거	대화-queue-artifact · 편집 action · activity · capability · Mock operation 각 test 통과. 운영용 durable worker와 process restart recovery는 미구현 범위

독립 prototype이다. 편집 기능은 기존 writer를 재사용했고, 계정-게시물 운영 action은 Mock gateway에서 실행 흐름과 안전 경계만 검증했다. 실제 예약·발행·삭제는 수행하지 않았다.

